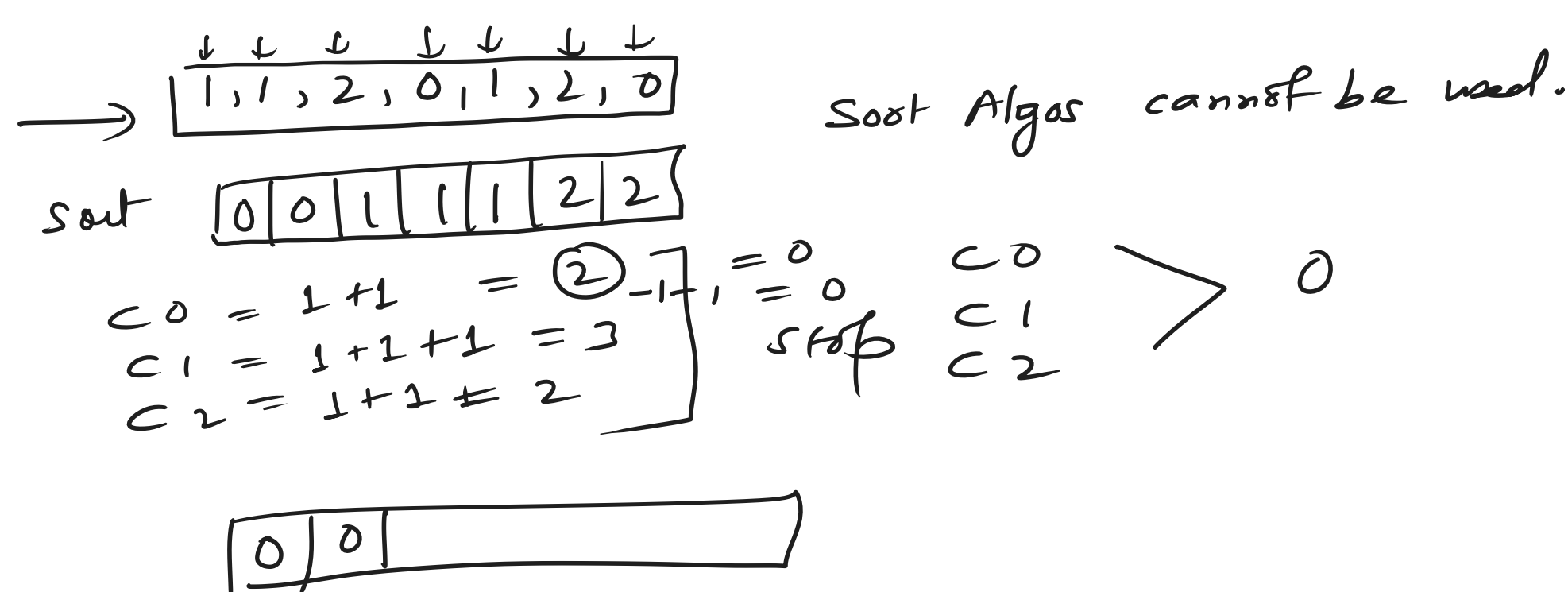
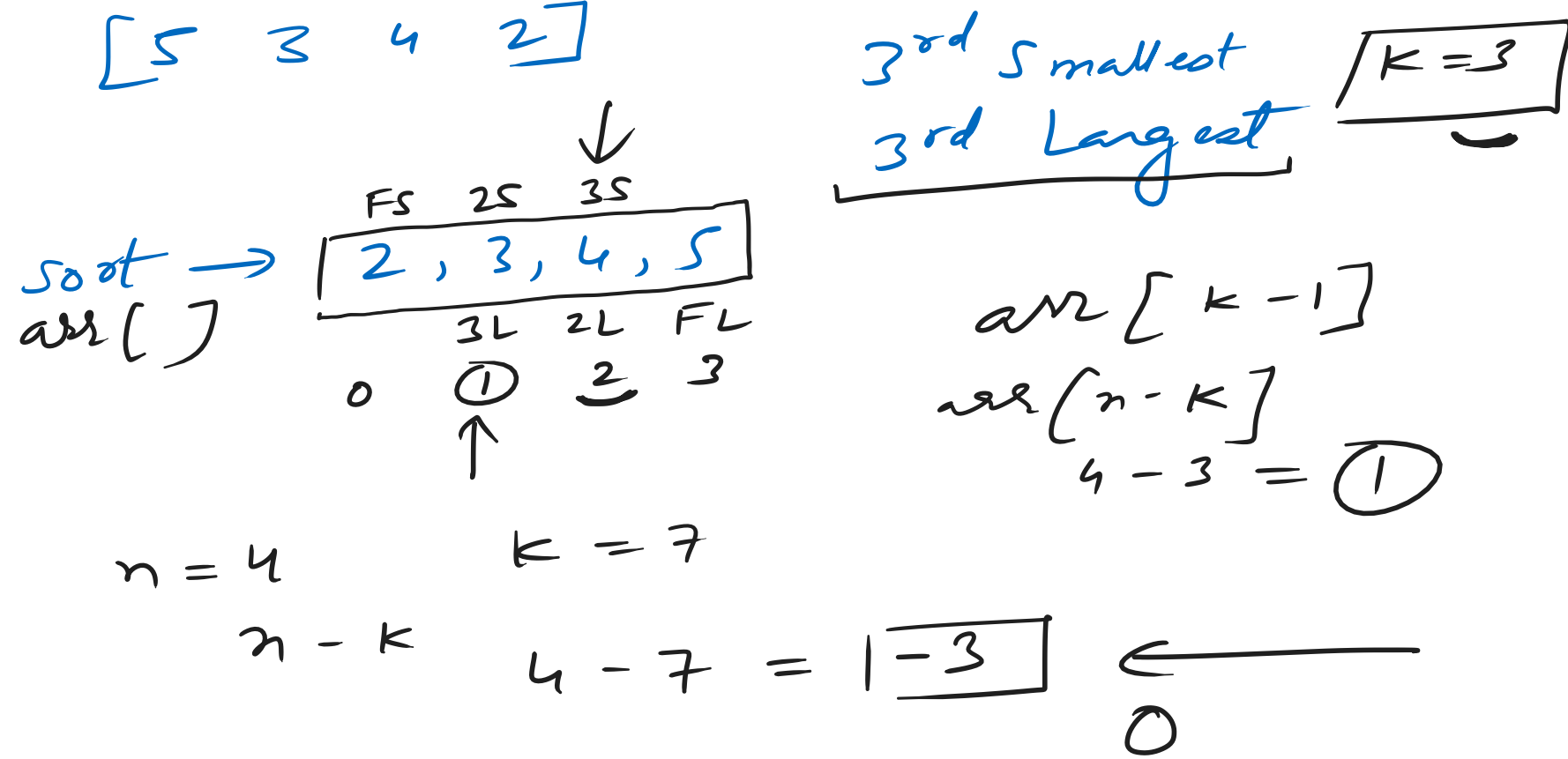
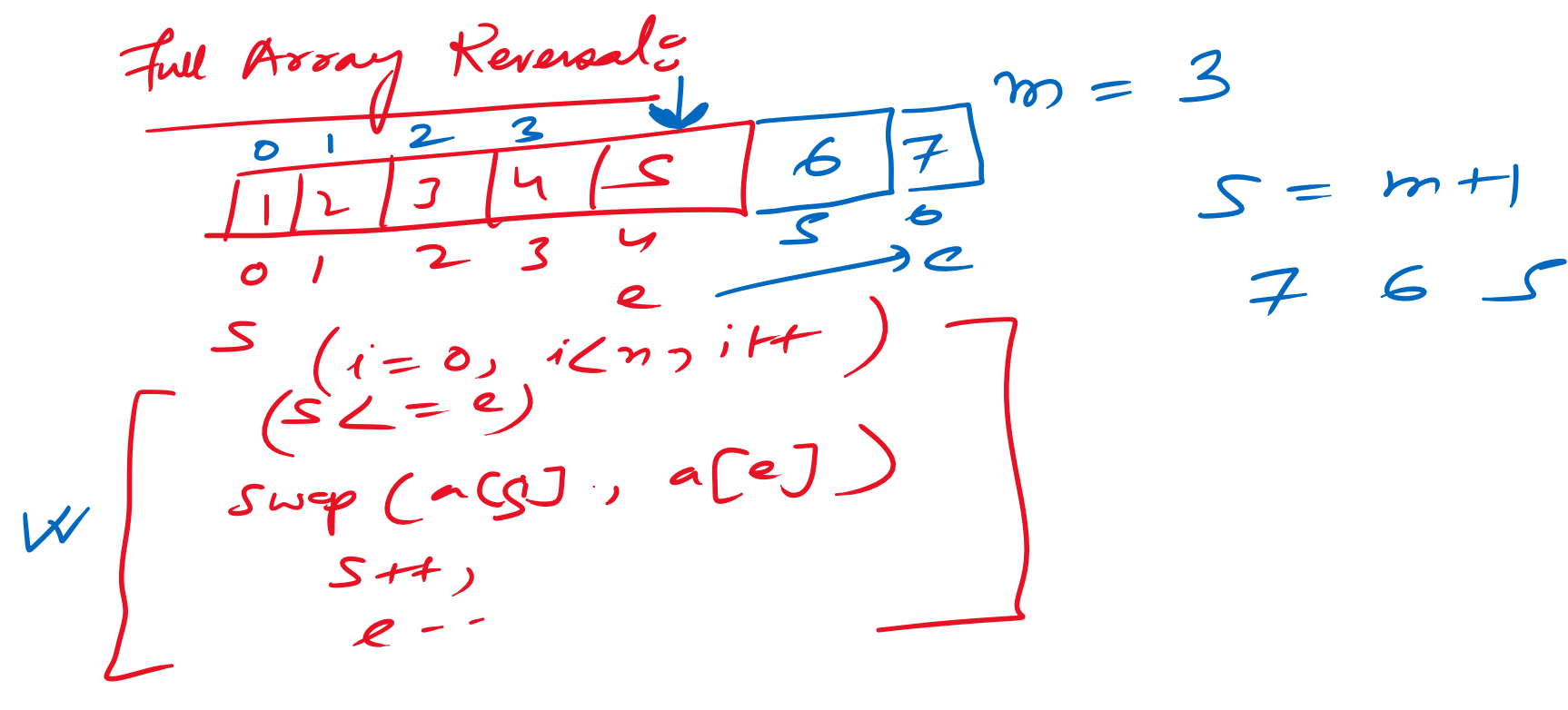




$\Rightarrow$ Data Structures & Algorithms : Problem

Store $\rightarrow$ Data

Kitchen	Dining	Bedroom
Cupb	Store	6
W1	W2	W3
Bedroom	1/2	1/2

- $\Rightarrow$ Find Them
- $\Rightarrow$ Efficient
- Less time
- $\Rightarrow$ Less money
- $\Rightarrow$ Less effort

Haphazard / ugly

Shoes	Flower	Bed
T-shirt	call sheet	
Carrot	Share chair	
Sugar		
Nice sofa	Food	

- $\Rightarrow$ Difficult to find
- $\Rightarrow$ More time
- $\Rightarrow$ More money

* Linear Data Structures

1. Arrays $\rightarrow$ 1D, 2D, MD
2. Stacks $\rightarrow$ LIFO
3. Queues $\rightarrow$ FIFO
4. Linked Lists
 - $\rightarrow$ Singly Linked List
 - $\rightarrow$ Doubly Linked List
 - $\rightarrow$ Circular Linked List
5. Priority Queue
6. STL
Standard Template Library

* Non-Linear Data Structures

1. Trees
 2. Tries
 3. Heaps (CBT)
 4. Graphs
 5. Binary Search Tree
 6. Balanced BST - AVL
- Adelson Velsky Landies
 (1962)
 Process of Storing Information

Algorithms:

Searching :

- ✓ (i) Linear Search
- (ii) Jump Search
- ✓ (iii) Binary Search ***
- ✓ (iv) Recursive Binary Search
- (v) Interpolation Search

Sorting:

- (i) Bubble Sort
- (ii) Selection Sort
- (iii) Insertion Sort
- (iv) Heap Sort
- *** (v) Merge Sort
- (vi) Quick Sort
- (vii) Count Sort
- (viii) Radix Sort

*** Time Complexity ***

Backtracking with Recursion $\rightarrow$

- * N Queen Problem
- * Rat In A Maze
- * Sudoku Solver
- * Subsets of a String

Dynamic Programming $\rightarrow$

- (i) Recursion
- (ii) Memoisation
- (iii) Tabulation

(iv) Space Optimization

* Stacks $\rightarrow$ Last In First Out

- * push() 1, 2, 3, 4, 5, 6
 - * pop() $\rightarrow$ 6, 5, 4, 3, 2, 1
 - peek() * top() $\rightarrow$ Topmost element
 - * size() $\rightarrow$ No. of elements
 - * empty() $\rightarrow$ size = 0
- Wood piece
-
- $top == \text{max size}$
 $\rightarrow$ stack overflow top $\rightarrow$ index
 $\rightarrow$ stack underflow
 $top == -1$
- [top = 0]
 [-1]

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

L T M

Upper Triangular Matrix:

- (0, 0), (0, 1), (0, 2), (0, 3)
- (1, 1), (1, 2), (1, 3)
- (2, 2), (2, 3), (3, 3)

[r <= c]